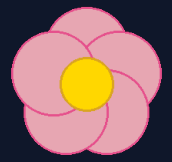
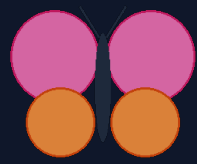
AURA MUSIC: Designing a Lightweight Music Streaming Application

Course Assignment 1: ICT, Computer Systems & System Architecture Proposal



Presenter: Balgyn Baikozhayev

Lead System Architect | University ICT Assignment Defense



ig: @balgynnbb

✦ ICT SYSTEM PROPOSAL SCOPE ✦

Defining Our System Architecture Proposal ('Aura Music')

🎯 Assignment 1 Core Objectives

• System Conceptualization: Propose a lightweight, ICT-enabled music streaming platform ('Aura Music').

• Technical Integration: Synthesize hardware, networks, polyglot storage, and software services.

• Standards Alignment: Embed ISO/IEC 27001 security controls and GDPR data privacy by design.

• Architectural Blueprints: Deliver visual system diagrams, relational ER schemas, and UI wireframes.

⚡ Proposed Application Features ('Aura Music')

• Low-Latency Audio Streaming: Instant playback via HLS chunking and CDN edge caching (<100ms).

• Relational Catalog & Search: PostgreSQL schema supporting song metadata, playlists, and media specs.

• Background Activity Logging: Lightweight event messaging queue for playback telemetry.

• Personal Listening Stats: Automated batch aggregations generating user profile stats (Top 0.001% badges).

✦ RESEARCH & ICT BASICS ✦

How Four ICT Pillars Power Our Streaming Application

1. Hardware & Compute

Cloud compute instances (AWS EC2 / GCP GCE) running microservice containers.

2. Networks & CDNs

Global CDN edge caching (Cloudflare) for instant audio delivery under 100ms.

3. Polyglot Storage

PostgreSQL for ACID metadata + Amazon S3 object storage for raw MP3/AAC audio files.

4. Software Services

Decoupled application microservices (User, Music Catalog, Streaming, Playlist) in Docker.

✦ GOVERNANCE & COMPLIANCE ✦

Safeguarding Information Security and User Data Privacy

🔒 ISO/IEC 27001:2022 (ISMS Framework)

• Logical Access Control (A.5.15): JWT OAuth 2.0 bearer tokens authenticate all API requests.

• Cryptographic Protection (A.8.24): TLS 1.3 encryption in transit & AES-256 for passwords at rest.

• Event Logging (A.8.15): Immutable security audit logs for authentication attempts.

• Operational Redundancy (A.8.14): Multi-AZ database replication for high availability.

🇪🇺 GDPR (General Data Protection Regulation)

• Privacy by Design (Article 25): Telemetry schemas automatically encrypt PII fields before logging.

• Lawfulness & Consent (Article 6): Explicit user opt-in for personal listening stats.

• Right to Erasure (Article 17): Automated scripts wipe user listening history upon account deletion.

• Breach Notification (Article 33): Automated incident detection triggers 72-hour notifications.

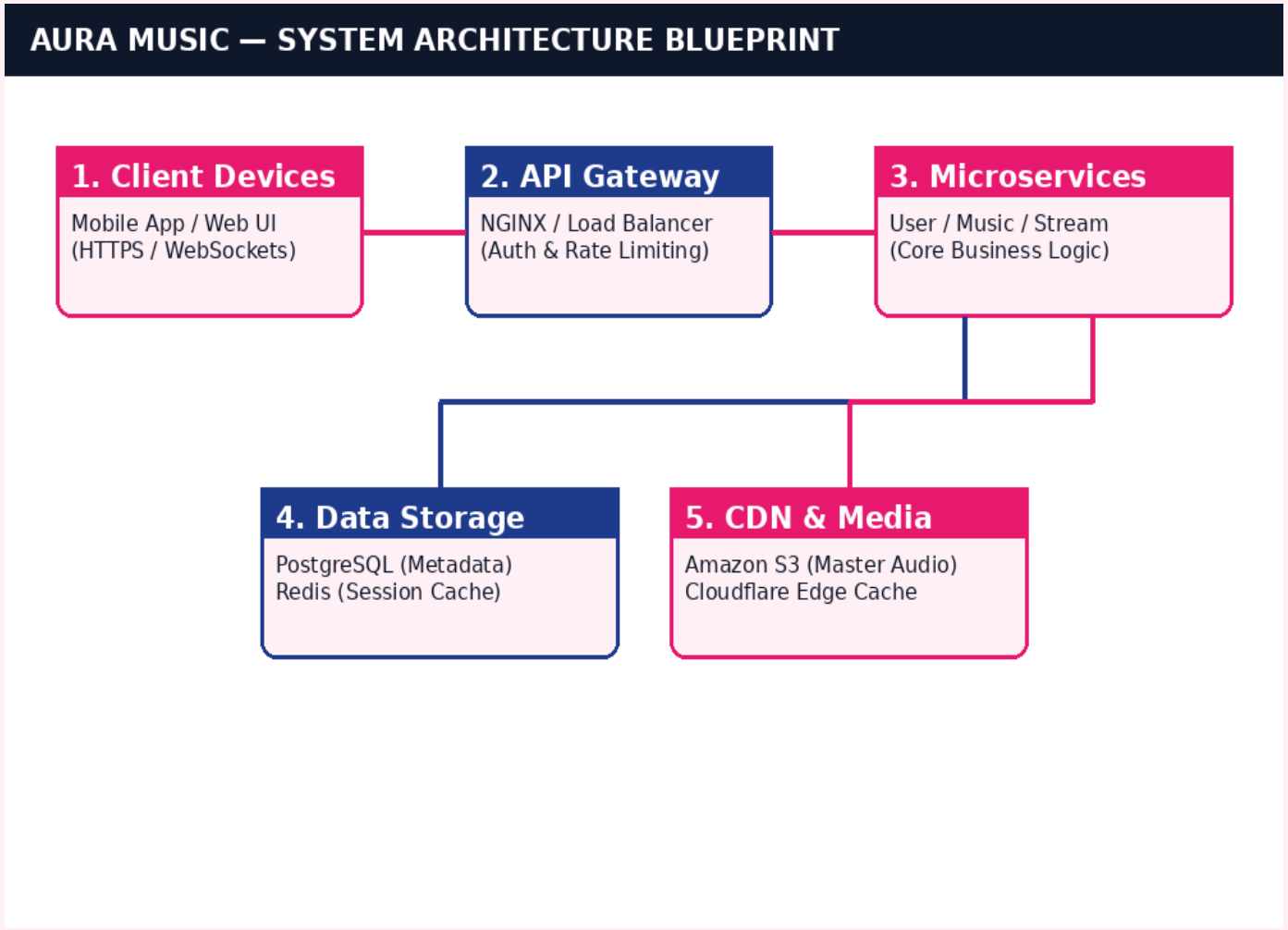
✦ SYSTEM COMPONENTS ✦

Modular Microservices Architecture Selection

Component Layer	Selected Technology	Primary Architectural Responsibility
Client Application	iOS / Android / Web (React)	Renders player UI, handles local HLS audio buffer.
API Gateway & Auth	NGINX / Kong API Gateway	JWT Token verification, rate limiting, request routing.
Application Microservices	Node.js / Python Services	User management, Music Catalog search, Playlist logic.
Databases & Cache	PostgreSQL + Redis Cache	ACID transactional data (users) + high-speed token caching.
Media Storage & CDN	Amazon S3 + Cloudflare CDN	Stores master audio binaries & caches tracks globally.

✦ ARCHITECTURE BLUEPRINT ✦

System Architecture Diagram: End-to-End Blueprint



Architecture Workflow Key

- 1. Client Layer: Mobile app sends authenticated HTTPS requests.
- 2. API Gateway: Validates JWT tokens & routes to targeted microservices.
- 3. Application Services: User Service, Music Service, and Streaming Service execute business logic.
- 4. Database & Storage: Queries PostgreSQL for metadata & Redis for fast cache hits.
- 5. CDN Delivery: Streaming Service fetches audio binaries from CDN edge servers for instant playback.

✦ DATA MANAGEMENT & ER SCHEMA ✦

Data Management: Relational Entity-Relationship (ER) Schema

RELATIONAL DATABASE SCHEMA — ER DIAGRAM



🏷 Relational Schema Breakdown

• `SONG` Table (Primary Entity): Stores track metadata (title, duration, genre, streaming_url) with `song_id` as Primary Key (PK).

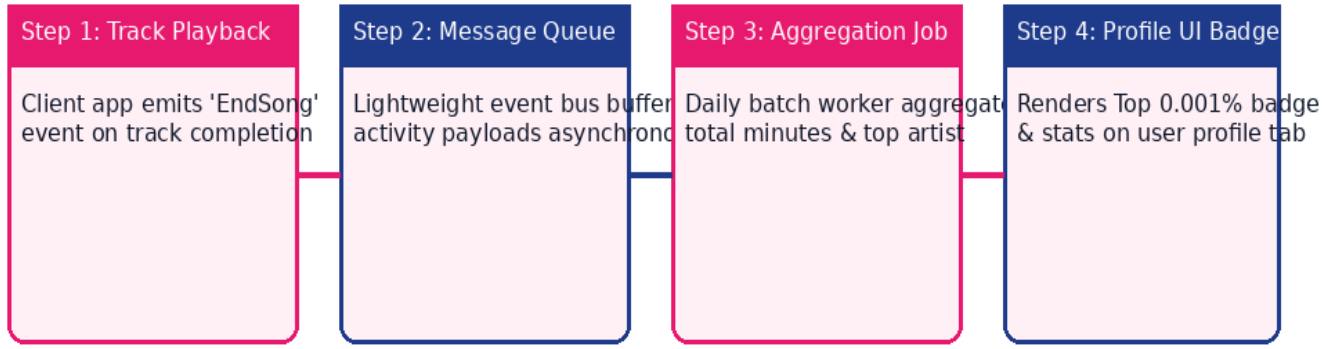
• `PLAYLIST_SONG` (Junction Table): Connects playlists and songs in a many-to-many relationship using `playlist_id` (FK) and `song_id` (FK).

• `MEDIA` Table (Audio Specs): Stores technical file details (file_url, mime_type, file_size) linked to `song_id` as Foreign Key (FK).

✦ FEATURE DESIGN ✦

Data Feature: Simple Activity Logging & Personal Listening Stats

LISTENING STATS & ACTIVITY LOGGING PIPELINE



Listening Stats Pipeline

- 1. Playback Event: Client app completes a song play and emits an 'EndSong' event payload.
- 2. Background Logging: Activity events pass through a lightweight message queue without slowing down live playback.
- 3. Aggregation Job: Daily batch scripts calculate total user minutes streamed, top artists, and top genres.
- 4. Personal Summary UI: Renders personalized stats badges directly in the app's user profile tab.

✦ UX & IDENTITY ✦

UX Psychology: Turning Database Telemetry into Shared Social Identity

🗄️ Cold Raw Database JSON Log

```
{
  "event": "EndSong",
  "user_id": 89421,
  "artist": "Olivia Rodrigo",
  "track": "vampire",
  "duration_ms": 219000,
  "timestamp": "2026-03-14T12:04:11Z"
}
```

🎨 Gamified Social Identity Card

• Emotional Transformation: Converts raw timestamps into badges of honor ('You're in the Top 0.001% of Olivia Rodrigo fans!').

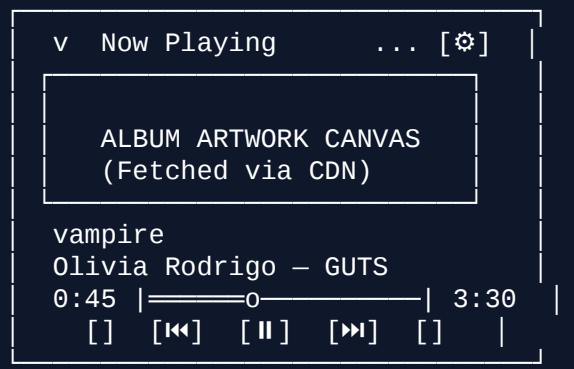
• Psychological Drivers: Leverages Self-Expression, Social Proof, and Music Nostalgia.

• Engineered for Virality: Native 9:16 vertical story format designed for 1-tap sharing to Instagram & TikTok.

✦ UI WIREFRAMES ✦

Wireframe Artifact 1: Standard Player UI & Backend API Mapping

🎨 Player UI Sketch Wireframe



🛠 UI Component & Backend API Callout Mapping

• Album Cover Art: Dynamic image load from CDN edge servers (`cdn.auramusic.com/art/vampire.png`).

• Metadata Label: Populated via GET `/api/v1/songs/{id}` endpoint returning title, artist, album.

• Interactive Seek Bar: Displays real-time HLS playback buffer position.

• Like Button (Heart): Triggers POST `/api/v1/users/me/likes` updating PostgreSQL database.

✦ REQUEST FLOW ARCHITECTURE ✦

Wireframe Artifact 2: Request Lifecycle (Synchronous vs. Asynchronous)

⚡ Synchronous Flow (Direct Audio Playback)

• Path: Client App → API Gateway → Streaming Service → CDN Edge Server → Client.

• Purpose: Delivers low-latency audio playback under 100ms.

• Execution: Direct HTTP/REST call. The client waits for audio chunks to start streaming immediately.

Asynchronous Flow (Background Activity Logging)

• Path: Client App → Message Queue → Analytics Worker → Aggregated Database.

• Purpose: Non-blocking background event logging for user stats & recommendations.

• Execution: Decoupled event emission (`EndSong`). Does not block or delay live music playback.

✦ UI WIREFRAMES ✦

Wireframe Artifact 3: 9:16 Listening Stats Story Card (Olivia Rodrigo Example)

9:16 Story Frame Wireframe

```
┌──────────────────────────────┐
│ [===] [===] [===] (Story Bar) │
│ YOUR TOP ARTIST OF THE YEAR  │
│ #1 OLIVIA RODRIGO            │
│ ┌──────────────────────┐     │
│ │   HERO ARTIST GRAPHIC │     │
│ └──────────────────────┘     │
│ 🏆 Top 0.001% Global Listener │
│ ⏱️ 12,450 Minutes Streamed    │
│  Top Track: 'vampire'       │
│ [ 📤 SHARE TO INSTAGRAM ]    │
└──────────────────────────────┘
```

🏷️ Rubric Compliance & Visual Elements

- Aspect Ratio: 9:16 vertical smartphone layout tailored for Instagram Stories & TikTok.
- Personalized Badge: 'Top 0.001% of Olivia Rodrigo fans' calculated via batch metrics.
- Key Metrics Display: Highlights total minutes listened (12,450 mins) and top song ('vampire').
- Social Share CTA: One-tap export button triggers OS native share sheet.

✦ ACADEMIC REFERENCES ✦

Academic & Technical Web References

📚 Formal Bibliography & Web Attributions

[1] GeeksforGeeks. (2024). Design Spotify Premium | System Design Architecture Blueprint. GeeksforGeeks. https://www.geeksforgeeks.org/design-spotify-system-design/

[2] Spotify Engineering. (2020). Spotify Unwrapped: How we brought you a decade of data. Spotify R&D Blog. https://engineering.atspotify.com/

[3] Spotify Engineering. (2016). Spotify's Event Delivery – Life in the Cloud. Spotify R&D Blog. https://engineering.atspotify.com/

[4] Spotify Engineering. (2015). How Spotify Scales Apache Storm. Spotify R&D Blog. https://engineering.atspotify.com/

[5] GDPR.eu. (2018). What is GDPR, the EU's new data protection law? General Data Protection Regulation Guide. https://gdpr.eu/

[6] ISO/IEC. (2022). ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — ISMS Requirements. International Organization for Standardization.

[7] IBM. (2023). What Is Cloud Computing? Service Models & Infrastructure Architecture. IBM Education. https://www.ibm.com/topics/cloud-computing

✦ DEFENSE CONCLUSION ✦

Thank You! Questions & Architectural Discussion

□ Architectural Defense Summary

□ Scalable Architecture: Modular microservices paired with PostgreSQL & CDN edge caching.

□ Standards Compliance: ISO/IEC 27001 access security & GDPR Privacy by Design.

□ Engaging Feature UX: Background activity logging transforming telemetry into viral stats badges.

Presented by Balgyn Baikozhayev | ig: @balgynnbb